# APPLICATION NOTE

## Linux 系统_ATBM_WIFI 相关_FAQ

**ATBM603X**

**1x1 802.11b/g/n Wi-Fi 芯片**

# Table of contents


**1 STA MODE**

1.1 内核需要打开如下宏用于使能 iwconfig，iwpriv，iwlist 命令

1.2 启动流程

(1) 加载 cfg80211.ko

(2) 加载 atbm_wifi.ko

(3) 启动 wpa_supplicant

(4) 启动 dhcpc

1.3 连接路由方法

(1) 通过 wpa_cli 连接

(2) 通过修改 wpa_supplicant.conf 连接【常用】

1.4 问题分析

(1) 连接不上路由器

(2) Ifconfig wlan0 up 不起来

(3) Wpa_supplicant 使用 WEXT 接口，Wpa_cli 扫描执行一直失败

**2 AP MODE**

2.1 内核需要打开如下宏用于使能 iwconfig，iwpriv，iwlist 命令

2.2 启动流程

(1) 加载 cfg80211.ko 见 STA MODE 部分

(2) 加载 atbm_wifi.ko 见 STA MODE 部分

(3) 启动 hostapd

(4) 启动 dnsmasq/dhcpd 二选一，建议使用 dnsmasq

2.3 注意事项

(1) hostapd 启动失败

(2) 客户端连接不上热点

(3) 设置 n 模式，HT40

(4) 客户端获取不到 IP


AN9310
Doc Rev: 1.7
Released:2020-05-08

(5) nl80211:set key fail error = -22 ....17
(6) hostapd.conf 设置隐藏 ssid 导致驱动挂掉....17
(7) 执行了命令，hostapd 重载配置无效....18
(8) hostapd 运行起来，手机连接上了但是没法建立 TCP 连接....19

**3 BRIDGE MDOE....21**

3.1 内核需要打开如下宏用于使能 iwconfig，iwpriv，iwlist 命令....21
3.2 功能说明....21
3.3 启动流程....21
(1) 加载 cfg80211.ko 见 STA MODE 部分....21
(2) 加载 atbm_wifi.ko 见 STA MODE 部分....21
(3) 加载 llc.ko....22
(4) 加载 stp.ko....22
(5) 加载 bridge.ko....22
(6) 启动 wlan0 接口....22
(7) 启动 wlan1/p2p0 接口....22
(8) 设置网桥接口 br0....22
(9) 启动网桥接口 br0....23
(10) 启动 wpa_supplicant....23
(11) 启动 dhcpc....23
(12) 启动 hostapd....23
3.4 注意事项....24
(1) 网桥接口 br0 起不来....24
(2) 使用 brctl 没法添加接口道 br0....24
(3) 使用桥接以后发现获取 IP 地址很慢....25
(4) 将 p2p0 加入桥接以后想修改网口模式，但是修改失败....25

**4 MONITOR MODE....27**

4.1 内核需要打开如下宏用于使能 iwconfig，iwpriv，iwlist 命令....27
4.2 启动 monitor 接口：....27
4.3 切换监听信道....27
4.4 关闭 monitor 接口：....27

| Author | version | change | 日期 |
|---|---|---|---|
| yuzhihuang | Init v1.0 | Init | |
| Yuzhihuang | v1.1 | Add monitor func | |
| yuzhihuang | V1.2 | 增加桥接模式下获取 ip 地址慢的解决方法 | |
| Yuzhihuang | V1.3 | 增加 hostapd 重载配置的方法 | 2019-12-31 |
| Yuzhihuang | V1.4 | 增加需要打开内核相关的宏的说明 | 2020-03-03 |
| Yuzhihuang | V1.5 | 增加hostapd运行异常说明与解决方法 | 2020-03-20 |
| Yuzhihuang | V1.6 | 模板 | 2020-04-29 |
| Yuzhihuang | V1.7 | 内核宏没开完全导致的通信问题 | 2020-05-08 |

# 1 STA MODE

## 1.1 内核需要打开如下宏用于使能 iwconfig，iwpriv，iwlist 命令

CONFIG_CFG80211_WEXT=y
CONFIG_WIRELESS=y
CONFIG_WIRELESS_EXT=y
CONFIG_WEXT_CORE=y
CONFIG_WEXT_PROC=y
CONFIG_WEXT_PRIV=y

## 1.2 启动流程

### (1) 加载 cfg80211.ko

insmod cfg80211.ko

```
[/ext/demo]## insmod /lib/modules/cfg80211.ko
cfg80211: Calling CRDA to update world regulatory domain
[/ext/demo]##
```

### (2) 加载 atbm_wifi.ko

insmod atbm_wifi.ko

```
[/ext/demo]## insmod /ext/lib/modules/atbm_wifi.ko
atbm_init_firmware
xxxx minstrel ht init
xxxx minstrel ht init
SVN_VER=901,DPLL_CLOCK=1,BUILD_TIME=[===USB-APOLLO==
[no platform]:platform set power [1]
Probe called
self->tx_hwChanId 0
Allocated hw_priv @ c6355300
atbm_before_load_firmware++
++++++++++++++++1.0V++++++++++++++++++
atbm_start_load_firmware++
atbm_start_load_firmware: used firmware.h=
atbm_start_load_firmware: START DOWNLOAD ICCM=========
atbm_load_firmware_generic: addr 0: len 20000
atbm_start_load_firmware: START DOWNLOAD DCCM=========
atbm_load_firmware_generic: addr 800000: len 8000
atbm_after_load_firmware++
mdelay wait wsm_startup_done  !!
wsm_caps.firmwareCap 1 firmware used old-rate policy
apollo wifi WSM init done.
   Input buffers: 32 x 1632 bytes
   Hardware: 7.21
   WSM firmware [=MODEM=RF=ATHENA_B  2GHZ Jul 27 2018 11:46:49], ver: 5050, build: 2029, api: 1060, cap: 0x0001 Config[30004]  expection 900b8b4, ep0 cmd addr 901d1a0
##############ERROR: hmac cmd not use ep0! hmac cmd use ep0!###################
atbm_firmware_init_check send data   (null)
apollo wifi : can't open /data/.mac.info
efuse data is [0x1,0x32,0x8,0xa,0x9,0x1b,0x0,0x0,0x0:0x0:0x0:0x0:0x0:0x0]
ELOG_INIT len 64
[atbm_wtd]:set wtd_probe = 1
usbcore: registered new interface driver atbm_wlan
[wtd] register.
[/ext/demo]## 
```

### (3) 启动 wpa_supplicant

要注意 wpa_supplicant.conf 文件的配置

一般 wpa_supplicant.conf 内容为：

ctrl_interface 参数要根据实际去修改

**注意：**

**/ext/demo/run/wpa_supplicant** 这个目录一定要存在，不存在手动创建一个。

ctrl_interface=/ext/demo/run/wpa_supplicant

该参数代表如果配置内容有变化会更新

update_config=1

启动的方式为：

wpa_supplicant –Dnl80211 -i wlan0 -c /ext/demo/run/wpa_supplicant.conf &

如果内核支持 wext 也可以是：

wpa_supplicant –Dwext -i wlan0 -c /ext/demo/run/wpa_supplicant.conf &

启动以后要确认几件事情：右图说明一直在扫描周围的热点

1、 wpa_supplicant 正常运行

2、 ifconfig 可以看到 wlan0 接口

```
 104 root         0 SW   [usb_atbm_bh]
 106 root      2816 S    wpa_supplicant -Dwext -i wlan0 -c /ext/demo/run/wpa_
 112 root      2200 R    ps
[/ext/demo]## atbm_hw_scan:if_id(0)
ifcohw_priv->bStartTx 0
hw_priv->scan.status 0
atbm_scan_work:end(0)
nfig
eth0      Link encap:Ethernet  HWaddr 08:00:20:49:32:60
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:90 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

wlan0     Link encap:Ethernet  HWaddr 00:12:34:6B:3E:8B
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

[/ext/demo]## ■
```

```
hw_priv->bStartTx 0
hw_priv->scan.status 0
atbm_scan_work:end(0)
atbm_hw_scan:if_id(0)
hw_priv->bStartTx 0
hw_priv->scan.status 0
atbm_scan_work:end(0)
atbm_hw_scan:if_id(0)
hw_priv->bStartTx 0
hw_priv->scan.status 0
atbm_scan_work:end(0)
atbm_hw_scan:if_id(0)
hw_priv->bStartTx 0
hw_priv->scan.status 0
atbm_scan_work:end(0)
atbm_hw_scan:if_id(0)
hw_priv->bStartTx 0
hw_priv->scan.status 0
atbm_scan_work:end(0)
atbm_hw_scan:if_id(0)
hw_priv->bStartTx 0
hw_priv->scan.status 0
atbm_scan_work:end(0)
atbm_hw_scan:if_id(0)
hw_priv->bStartTx 0
hw_priv->scan.status 0
atbm_scan_work:end(0)
```

### (4) 启动 dhcpc

udhcpc –i wlan0 &

## 1.3 连接路由方法

### (1) 通过 wpa_cli 连接

a) 看 wpa_supplicant.conf 中:

ctrl_interface=/var/run/wpa_supplicant

b) wpa_cli -p/var/run/wpa_supplicant -iwlan0

//如果 wpa_supplicant.conf 没有保存 network 的话就为 0，有 2 个 network 的话就为 2

>add_network 0 // 添加一个网络号

>set_network 0 ssid '"TP-LINK_64F8"' // 设置要连接的 ssid

OK

>set_network 0 priority 9 // 设置优先级

OK

>set_network 0 key_mgmt WPA_PSK // 设置要连接的路由器加密方式

OK

>set_network 0 psk '"12345678"' // 设置路由器密码

OK

>enable_network 0 // 使能网络，设置了这一步以后会自动去连接路由器了

OK

### (2) 通过修改 wpa_supplicant.conf 连接【常用】

常用配置说明：

```
# 请不要修改下面这一行内容，否则将不能正常工作
ctrl_interface=/var/run/wpa_supplicant

# 确保只有 root 用户能读取 WPA 的配置
ctrl_interface_group=0

# 使用 wpa_supplicant 来扫描和选择 AP
ap_scan=1

# 简单的情形：WPA-PSk 密码验证方式，PSK 是 ASCII 密码短语，所有合法的加密方式都允许连接
network={
   ssid="simple"
   psk="very secret passphrase"
   # 优先级越高，就能越早匹配到。
   priority=5
}

# 与前面的设置相同，但要求对特定的 SSID 进行扫描（针对那些拒绝广播 SSID 的 AP 也就我们常说的隐藏 ap）
network={
   ssid="second ssid"
   scan_ssid=1
   psk="very secret passphrase"
   priority=2
}

# 仅使用 WPA-PSK 方式。允许使用任何合法的加密方式的组合
network={
   ssid="example"
   proto=WPA
   key_mgmt=WPA-PSK
   pairwise=CCMP TKIP
   group=CCMP TKIP WEP104 WEP40
   psk=06b4be19da289f475aa46a33cb793029d4ab3db7a23ee92382eb0106c72ac7bb
   priority=2
}

# 明文连接方式（不使用 WPA 和 IEEE802.1X）
network={
   ssid="plaintext-test"
```

```
    key_mgmt=NONE
}

# 共享 WEP 密钥连接方式（不使用 WPA 和 IEEE802.1X）
network={
    ssid="static-wep-test"
    key_mgmt=NONE
    # 引号包含的密钥是 ASCII 密钥
    wep_key0="abcde"
    # 没有引号包含的密钥是十六进制密钥
    wep_key1=0102030405
    wep_key2="1234567890123"
    wep_tx_keyidx=0
    priority=5
}

# 共享 WEP 密钥连接方式（无 WPA 和 IEEE802.1X），使用共享密钥 IEEE802.11 验证方式
network={
    ssid="static-wep-test2"
    key_mgmt=NONE
    wep_key0="abcde"
    wep_key1=0102030405
    wep_key2="1234567890123"
    wep_tx_keyidx=0
    priority=5
    auth_alg=SHARED
}

# 在 IBSS/ad-hoc 网络中使用 WPA-None/TKIP
network={
    ssid="test adhoc"
    mode=1
    proto=WPA
    key_mgmt=WPA-NONE
    pairwise=NONE
    group=TKIP
    psk="secret passphrase"
}
```

默认的 wpa_supplicant.conf 的内容为：

```
ctrl_interface=/ext/demo/run/wpa_supplicant
update_config=1
```

修改后的 wpa_supplicant.conf 的内容为：

```
ctrl_interface=/ext/demo/run/wpa_supplicant
update_config=1

network={
        ssid="TP-LINK_64F8"
        psk="12345678"
}
```

修改完配置需要重启 wpa_supplicant 应用：

killall wpa_supplicant

wpa_supplicant –Dnl80211 -i wlan0 -c /ext/demo/run/wpa_supplicant.conf &

下图是已经连接上路由器但是没有去获取 IP

```
[/ext/demo]##
[/ext/demo]## wpa_supplicant -Dwext -i wlan0 -c /ext/demo/run/wpa_supplicant.con
f  &
[/ext/demo]## Successfully initialized wpa_supplicant
rfkill: Cannot oatbm_add_interface: vif->type 2, vif->p2p 0, addr 00:12:34:6b:3e:8b
pen RFKILL control device
[STA] Slot time :20 us.
atbm_hw_scan:if_id(0)
hw_priv->bStartTx 0
hw_priv->scan.status 0
atbm_scan_work:end(0)
wlan0: Trying toieee80211_mgd_auth:(34:96:72:f4:8a:8d)
 associate with ieee80211_work_work:start work ch(6)
34:96:72:f4:8a:8[WSM] Issue join command.
d (SSID='altobeaatbm_join_work:ch(6),chtype(0),if_id(0)
mSZ_2.4g' freq=2437 MHz)
ioctl[SIOCSIWFREQ]: Device or resource busy
wlan0: Association requesatbm_join_work:enable combination mode
atbm_join_work:end
t to the driver failed
ieee80211_start_connecting_work:bssid(34:96:72:f4:8a:8d)
minstrel_ht_update_caps,739:IEEE80211_HT_CAP_SUP_WIDTH_20_40
atbm_config: push queue chantype_change_work pending,joinstatus(3),chatype(2)
[STA] arp ip filter enable: 0
atbm_channel_type_change_work:chatype(2),channelNumber(6)
atbm_channel_type_change_work  426
atbm_channel_type_change_work  430
atbm_bss_info_changed:priv->htcap(1)
[STA] Slot time :9 us.
wlan0: Associated with 34:96:72:f4:8a:8d
down_eap_rate
ieee80211_work_work:reset work ch(6)
ieee80211_work_work:start work ch(6)
ieee80211_wk_connecting:connecting.tries(1)
down_eap_rate
wlan0: WPA: Key negotiation completed with 34:96:72:f4:8a:8d [PTK=CCMP GTK=TKIP]
wlan0: CTRL-EVENT-CONNECTED - Connection to 34:96:72:f4:8a:8d completed [id=3 id_str=]
ieee80211_wk_connecting:connecting.tries(2)
ieee80211_wk_connecting:connecting.tries(3)
ieee80211_wk_connecting:connecting.tries(4)
ieee80211_wk_connecting:connecting.tries(5)
ieee80211_wk_connecting:connecting.tries(6)
ieee80211_wk_connecting: time out
ieee80211_work_work:reset work ch(6)
ieee80211_connecting_work_done
```

如果发现获取不到 IP 地址需要重启 udhcpc

killall udhcpc

udhcpc –i wlan0 &

```
[/ext/demo]##
[/ext/demo]## udhcpc -i wlan0
udhcpc (v1.24.1) started
Setting IP address 0.0.0.0 on wlan0
Sending discover...
down_bootp_rate
<WIFI> rx ampdu ++
cancel dhcp_retry_work
Sending select for 192.168.1.120cancel dhcp_retry_work
...
down_bootp_rate
cancel dhcp_retry_work
Lease of 192.168.1.120 obtained, lease time 7200
Setting IP address 192.168.1.120 on wlan0
[STA] arp ip filter enable: 3
wsm_set_pm_indication
ieee80211_ifa_changed(wlan0):IPv4 enable,end_time(97210)
ieee80211_cancle_connecting_work,(34:96:72:f4:8a:8d)
Deleting routers
route: SIOCDELRT: No such process
Adding router 192.168.1.1
Recreating /data/resolv.conf
 Adding DNS server 192.168.1.1
 Adding DNS server 192.168.1.1
[/ext/demo]##
```

```
[/ext/demo]##
[/ext/demo]##
[/ext/demo]## ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:20:49:32:60
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:90 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

wlan0     Link encap:Ethernet  HWaddr 00:12:34:6B:3E:8B
          inet addr:192.168.1.120  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:586 errors:0 dropped:149 overruns:0 frame:0
          TX packets:5 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:60118 (58.7 KiB)  TX bytes:1070 (1.0 KiB)

[/ext/demo]## ping 192.168.1.1
PING 192.168.1.1wsm_handle_tx_data:WSM_HT_TX_WIDTH_40M
 (192.168.1.1): 56 data bytes
64 bytes from 192.168.1.1: seq=0 ttl=64 time=8.466 ms
64 bytes from 192.168.1.1: seq=1 ttl=64 time=1.987 ms
^C
```

## 1.4 问题分析

### (1) 连接不上路由器

a) 获取不到 IP

i. 判断 wpa_supplicant 是否已经运行

ii. 判断 WIFI 是否已经正常连接上,如上图 LOG

1. 如果没有正常连接上重启 wpa_supplicant 加上-d 参数

wpa_supplicant –Dnl80211 -i wlan0 -c /ext/demo/run/wpa_supplicant.conf -d&

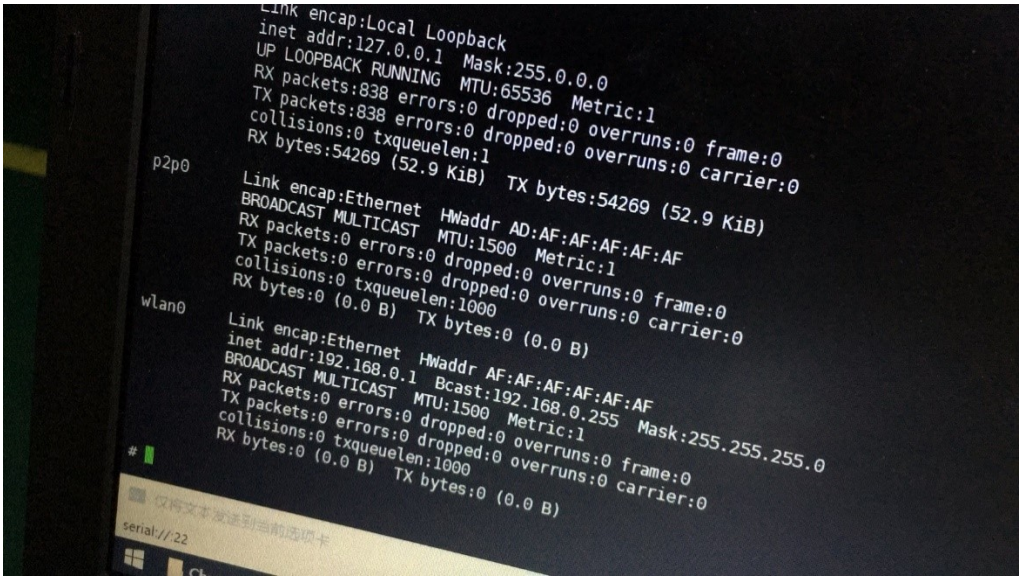
iii. 判断 udhcpc 是否已经运行

b) 连接不上 SSID 是中文的路由器,联系 FAE 拿 patch，需要修改 wpa_supplicant 源码

### (2) Ifconfig wlan0 up 不起来

执行 ifconfig wlan0 up 会报错 ifconfig:SIOCSIFHWADDR: Cannot assign requested address

执行 ifconfig –a:




此时的 mac 地址为组播地址。

IP 地址分为三类：广播，组播和单播。

**广播**就是：FF:FF:FF:FF:FF:FF。

**组播**：第一字节最后一位是 1，如 47:72:65:65:6e:00，

47 的最后一位是 1。

**单播**：第一字节最后一位是 0，如 48:72:65:65:6e:00。

把上面的地址改为 48 就不会出现这个问题了。

问题深入学习：

IEEE 802 定义 MAC 地址为

|<---------- 24 bit ---------->|←---------- 24 bit ---------->|

| ccccccug cccccccc cccccccc | xxxxxxxx xxxxxxx xxxxxxxx |

由 ug 控制 mac 地址类型：

u：

0：由 IEEE 指定 ID 统一管理

1：本地管理

g：

0：单播

1：多播

也就是 12 位 MAC 地址分为四类，由其中第二位决定

第二位为

0 | 4 | 8 | C ：（00）统一管理的单播 MAC

1 | 5 | 9 | D ：（01）统一管理的多播 MAC

2 | 6 | A | E ：（10）本地管理的单播 MAC

3 | 7 | B | F： （11）本地管理的多播 MAC

==========
========

由于针对 ADSL 路由等这样的网络终端，一般使用的都是 统一管理的单播 MAC

所以会判断 02:10:18:01:00:01 或者（11:01:18:00:00:30）为无效 MAC，导致无线等功能失效,或者网络连接失败等现象。

而对于 00:25:5E:08:DE:43 这样的 MAC 就被认为是有效的

## (3) Wpa_supplicant 使用 WEXT 接口，Wpa_cli 扫描执行一直失败

通过执行 wpa_cli scan 会打印：

ioctl[SIOCSIWSCAN]: Invalid argument

并且在初始化 wpa_supplicant 也发现如下打印：

WEXT: SIOCSIWAUTH(param 7 value 0x1) failed: Invalid argument)

**分析：**

由于 ATBM wifi 依赖 cfg80211 模块，wpa_supplicant 扫描命令会先下发给到 cfg80211，cfg80211 再传给驱动进行扫描。

经过检查发现 wpa_supplicant 正常绑定 socket ioctl 到 cfg80211，打印如下：

```
8        74 2f 09 09                                        t/__
9  CTRL_IFACE GLOBAL INTERFACE_ADD 'wlan0          wext    /var/tmp/wpa_supplicant/                '
0  Initializing interface 'wlan0' conf 'N/A' driver 'wext' ctrl_interface '/var/tmp/wpa_supplicant/' bridge 'N/A'
1  WEXT: cfg80211-based driver detected
2  wext: interface wlan0 phy: phy0
3  rfkill: Cannot open RFKILL control device
4  WEXT: RFKILL status not available
5  [08:00:10.820 info task VSP_HS_650_HStask.c:1874]wait eth0 dhcp ok 9
```

但是通过 ioctl 传输参数不正常，所以就怀疑是否是内核的宏没有开完全？

```
CONFIG_WIRELESS=y
CONFIG_WIRELESS_EXT=y
CONFIG_WEXT_CORE=y
CONFIG_WEXT_PROC=y
CONFIG_WEXT_PRIV=y
CONFIG_CFG80211=m
# CONFIG_NL80211_TESTMODE is not set
# CONFIG_CFG80211_DEVELOPER_WARNINGS is not set
# CONFIG_CFG80211_CERTIFICATION_ONUS is not set
CONFIG_CFG80211_DEFAULT_PS=y
# CONFIG_CFG80211_INTERNAL_REGDB is not set
CONFIG_CFG80211_CRDA_SUPPORT=y
# CONFIG_CFG80211_WEXT is not set
# CONFIG_LIB80211 is not set
CONFIG_MAC80211=m
CONFIG_MAC80211_HAS_RC=y
CONFIG_MAC80211_RC_MINSTREL=y
CONFIG_MAC80211_RC_MINSTREL_HT=y
# CONFIG_MAC80211_RC_MINSTREL_VHT is not set
CONFIG_MAC80211_RC_DEFAULT_MINSTREL=y
CONFIG_MAC80211_RC_DEFAULT="minstrel_ht"
# CONFIG_MAC80211_MESH is not set
/WEXT
```

经过检查内核 CONFIG_CFG80211_WEXT 宏没开，打开宏以后重新编译 cfg80211 以及 ATBM 驱动就可以正常使用了。

# 2 AP MODE

## 2.1 内核需要打开如下宏用于使能 iwconfig，iwpriv，iwlist 命令

CONFIG_CFG80211_WEXT=y
CONFIG_WIRELESS=y
CONFIG_WIRELESS_EXT=y
CONFIG_WEXT_CORE=y
CONFIG_WEXT_PROC=y
CONFIG_WEXT_PRIV=y

## 2.2 启动流程

### (1) 加载 cfg80211.ko 见 STA MODE 部分

### (2) 加载 atbm_wifi.ko 见 STA MODE 部分

### (3) 启动 hostapd

一般 hostapd.conf 为如下配置：ssid : atbm6022 password:12345678

```
interface=p2p0
#bridge=br0
ctrl_interface=/ext/demo/run/hostapd/
ctrl_interface_group=0
driver=nl80211
ieee80211n=1
ssid=ATBM_WIFI_AP
hw_mode=g
channel=9
macaddr_acl=0
auth_algs=1
ignore_broadcast_ssid=0
wpa=2
wpa_passphrase=12345678
wpa_key_mgmt=WPA-PSK
wpa_pairwise=CCMP
rsn_pairwise=CCMP
#ht_capab=[HT40-]
#beacon_int=200
wpa_group_rekey=0
max_num_sta=4
```

设置为 HT40 参数配置如下:

ht_capab=[HT40-][HT40+]

HT40 配置说明：

| chan/freq | ht_capab |
|---|---|
| 1~4 | [HT40+] |

| | |
|---|---|
| 5~7 | [HT40+]或者[HT40-]或者[HT40+][HT40-] |
| 7~13 | [HT40-] |
| 2380 | [HT40-] |
| 2504 | [HT40+] |

需要设置为不加密修改 wpa=0 即可

运行 hostapd：

hostapd -B /ext/demo/run/hostapd.conf &

需要调试加上-d：debug 的意思

hostapd -B /ext/demo/run/hostapd.conf -d&

### (4) 启动 dnsmasq/dhcpd 二选一，建议使用 dnsmasq

dnsmasq interface=wlan0 --no-daemon --no-resolv --no-poll --dhcp-range=192.168.100.2,192.168.100.254,12h &

**【注意】**

这么配置不需要使用 dnsmasq.conf,在系统中找到 dnsmasq.conf 直接删除。

## 2.3 注意事项

### (1) hostapd 启动失败

a) 没法设置 11 以上的信道

无法设置 11 信道是因为 cfg80211 做了限制导致，需要修改 cfg80211。

修改方式如下：

在 kernel/net/wireless/reg.c 237 行左右的位置

注释掉第一个箭头指向的枚举变量，可以将 hostapd 的信道设置为 12/13

注释掉第二个箭头指向的枚举变量，可以将 hostapd 的信道设置为 14

框框里面需要注意都为 40，代表 cfg80211 支持 HT40

```
/* We keep a static world regulatory domain in case of the absence of CRDA */
static const struct ieee80211_regdomain world_regdom = {
        .n_reg_rules = 5,
        .alpha2 =  "00",
        .reg_rules = {
                /* IEEE 802.11b/g, channels 1..11 */
                REG_RULE(2412-10, 2462+10, 40, 6, 20, 0),
                /* IEEE 802.11b/g, channels 12..13. */
                REG_RULE(2467-10, 2472+10, 40, 6, 20,
                        #if 0
                                NL80211_RRF_NO_IR
                        #endif
                        0),
                /* IEEE 802.11 channel 14 - Only JP enables
                 * this and for 802.11b only */
                REG_RULE(2484-10, 2484+10, 20, 6, 20,
                        #if 0
                                NL80211_RRF_NO_IR |
                                NL80211_RRF_NO_OFDM
                        #endif
                        0),
                /* IEEE 802.11a, channel 36..48 */
                REG_RULE(5180-10, 5240+10, 160, 6, 20,
                        NL80211_RRF_NO_IR),

                /* IEEE 802.11a, channel 52..64 - DFS required */
                REG_RULE(5260-10, 5320+10, 160, 6, 20,
                        NL80211_RRF_NO_IR |
                        NL80211_RRF_DFS),

                /* IEEE 802.11a, channel 100..144 - DFS required */
                REG_RULE(5500-10, 5720+10, 160, 6, 20,
                        NL80211_RRF_NO_IR |
                        NL80211_RRF_DFS),
```

b) Hostapd 为 14 信道只能运行 g 模式

需要修改 hostapd 源码，对应源码位置如下：

hostapd/src/ap/ hw_features.c 按照下图注释修改

```
int hostapd_select_hw_mode(struct hostapd_iface *iface)
{
    int i;

    if (iface->num_hw_features < 1)
        return -1;

    if ((iface->conf->hw_mode == HOSTAPD_MODE_IEEE80211G ||
         iface->conf->ieee80211n || iface->conf->ieee80211ac) &&
        iface->conf->channel == 14) {
        wpa_printf(MSG_INFO, "Disable OFDM/HT/VHT on channel 14");
        //iface->conf->hw_mode = HOSTAPD_MODE_IEEE80211B;
        //iface->conf->ieee80211n = 0;
        iface->conf->ieee80211ac = 0;
    }

    iface->current_mode = NULL;
    for (i = 0; i < iface->num_hw_features; i++) {
        struct hostapd_hw_modes *mode = &iface->hw_features[i];
        if (mode->mode == iface->conf->hw_mode) {
            iface->current_mode = mode;
            break;
        }
    }
```

## (2) 客户端连接不上热点

出现如下的错误：

```
dnsmasq: failed to load names from /etc/hosts: No such file or directory
Using interface p2p0 with hwaddr 00:12:34:4d:24:99 and ssid "atbm_6022"
random: Cannot read from /dev/random: Resource temporarily unavailable
random: Only 0/20 bytes of strong random data available from /dev/random
random: Not enough entropy pool available for secure operations
WPA: Not enough entropy in random pool for secure operations - update keys later when the first station connects
atbm_upload_beacon:atbm_clear_wpas_p2p_40M_ie
atbm_clear_wpas_p2p_40M_ie:filter index(0)
atbm_clear_wpas_p2p_40M_ie:htcap_ie == NULL,len(85),pkg_len(121)
[BH] wakeup.
[BH] wakeup.
[BH] wakeup.
atbm_start_ap:start.channel_type,(1),channelNumber(1)
[BH] wakeup.
[BH] wakeup.
```

说明是获取随机数失败：

具体分析过程请看:《hostapd 启动后有时候密码对了还是会连接不上_解决方案.docx》

**解决方法有两种：**

1.在运行 hostapd /wpa_supplicnat 直接执行如下命令

rm –f /dev/random

ln –s /dev/urandom /dev/random

2.修改 hostapd 源码：

src/crypto/random.c 下图位置

将

fd = open("/dev/random",O_RDONLY|O_NONBLOCK)

修改为

fd = open("/dev/usrandom",O_RDONLY|O_NONBLOCK)

```
225:     if (dummy_key_avail == sizeof(dummy_key))
226:         return 1; /* Already initialized - good to continue */
227:
228:     /*
229:      * Try to fetch some more data from the kernel high quality
230:      * /dev/random. There may not be enough data available at this point,
231:      * so use non-blocking read to avoid blocking the application
232:      * completely.
233:      */
234:     fd = open("/dev/random", O_RDONLY | O_NONBLOCK);
235:     //fd = open("/dev/urandom", O_RDONLY | O_NONBLOCK);
236:     if (fd < 0) {
237:         wpa_printf(MSG_ERROR, "random: Cannot open /dev/random: %s",
238:                    strerror(errno));
239:         return -1;
240:     }
241:
242:     res = read(fd, dummy_key + dummy_key_avail,
243:                sizeof(dummy_key) - dummy_key_avail);
244:     if (res < 0) {
245:         wpa_printf(MSG_ERROR, "random: Cannot read from /dev/random: "
246:                    "%s", strerror(errno));
247:         res = 0;
248:     }
249:     wpa_printf(MSG_DEBUG, "random: Got %u/%u bytes from "
250:                "/dev/random", (unsigned) res,
251:                (unsigned) (sizeof(dummy_key) - dummy_key_avail));
252:     dummy_key_avail += res;
253:     close(fd);
254:
255:     if (dummy_key_avail == sizeof(dummy_key)) {
256:         if (own_pool_ready < MIN_READY_MARK)
257:             own_pool_ready = MIN_READY_MARK;
258:         random_write_entropy();
```

## (3) 设置 n 模式，HT40

第一步，在编译源码的时候要确认 hostapd 支持高速率 CONFIG_IEEE80211N=y

第二步，配置 hostapd.conf 增加如下参数：

ieee80211n=1

ht_capab=[HT40-]

参数说明如下：

```
##### IEEE 802.11n related configuration ####################################

# ieee80211n: Whether IEEE 802.11n (HT) is enabled
# 0 = disabled (default)
# 1 = enabled
# Note: You will also need to enable WMM for full HT functionality.
# Note: hw_mode=g (2.4 GHz) and hw_mode=a (5 GHz) is used to specify the band.
#ieee80211n=1

# ht_capab: HT capabilities (list of flags)
# LDPC coding capability: [LDPC] = supported
# Supported channel width set: [HT40-] = both 20 MHz and 40 MHz with secondary
#       channel below the primary channel; [HT40+] = both 20 MHz and 40 MHz
#       with secondary channel above the primary channel
#       (20 MHz only if neither is set)
#       Note: There are limits on which channels can be used with HT40- and
#       HT40+. Following table shows the channels that may be available for
#       HT40- and HT40+ use per IEEE 802.11n Annex J:
#       freq            HT40-           HT40+
#       2.4 GHz         5-13            1-7 (1-9 in Europe/Japan)
#       5 GHz           40,48,56,64     36,44,52,60
#       (depending on the location, not all of these channels may be available
#       for use)
#       Please note that 40 MHz channels may switch their primary and secondary
#       channels if needed or creation of 40 MHz channel maybe rejected based
#       on overlapping BSSes. These changes are done automatically when hostapd
#       is setting up the 40 MHz channel.
# Spatial Multiplexing (SM) Power Save: [SMPS-STATIC] or [SMPS-DYNAMIC]
#       (SMPS disabled if neither is set)
# HT-greenfield: [GF] (disabled if not set)
# Short GI for 20 MHz: [SHORT-GI-20] (disabled if not set)
# Short GI for 40 MHz: [SHORT-GI-40] (disabled if not set)
# Tx STBC: [TX-STBC] (disabled if not set)
# Rx STBC: [RX-STBC1] (one spatial stream), [RX-STBC12] (one or two spatial
#       streams), or [RX-STBC123] (one, two, or three spatial streams); Rx STBC
#       disabled if none of these set
# HT-delayed Block Ack: [DELAYED-BA] (disabled if not set)
# Maximum A-MSDU length: [MAX-AMSDU-7935] for 7935 octets (3839 octets if not
#       set)
# DSSS/CCK Mode in 40 MHz: [DSSS_CCK-40] = allowed (not allowed if not set)
# 40 MHz intolerant [40-INTOLERANT] (not advertised if not set)
# L-SIG TXOP protection support: [LSIG-TXOP-PROT] (disabled if not set)
#ht_capab=[HT40-][SHORT-GI-20][SHORT-GI-40]
```

第三步，这一步看情况执行，如果没法运行起来 HT40，在修改 hostapd 源码，Hostapd/src/ap/hw_features.c 增加如下内容

```
static void ieee80211n_check_scan(struct hostapd_iface *iface)
{
    struct wpa_scan_results *scan_res;
    int oper40;
    int res;
    int fix40 = 1;

    /* Check list of neighboring BSSes (from scan) to see whether 40 MHz is
     * allowed per IEEE Std 802.11-2012, 10.15.3.2 */

    iface->scan_cb = NULL;

    scan_res = hostapd_driver_get_scan_results(iface->bss[0]);
    if (scan_res == NULL) {
        hostapd_setup_interface_complete(iface, 1);
        return;
    }else if(fix40){
        wpa_scan_results_free(scan_res);
        hostapd_setup_interface_complete(iface, 0);
        return;
    }

    if (iface->current_mode->mode == HOSTAPD_MODE_IEEE80211A)
        oper40 = ieee80211n_check_40mhz_5g(iface, scan_res);
    else
        oper40 = ieee80211n_check_40mhz_2g4(iface, scan_res);
```

## (4) 客户端获取不到 IP

第一步，确认 dnsmasq/udhcpd 绑定的 IP 地址是否和 ifconfig wifi 接口的 IP 地址在同一个网段

第二步，如果使用的/etc/dnsmasq.conf【/etc/udhcpd.conf】，注意 dnsmasq.conf【udhcpd.conf】内容不要和 dnsmasq【udhcpd】运行参数重复，特别是 IP 网段设置那一块。

### (5) nl80211:set key fail error = -22

```
16:37:29:272 Get randomness: len=16 entropy=0
16:37:29:286 random from os_get_random - hexdump(len=16): [REMOVED]
16:37:29:291 random_mix_pool - hexdump(len=20): [REMOVED]
16:37:29:343 random from internal pool - hexdump(len=16): [REMOVED]
16:37:29:343 mixed random - hexdump(len=16): [REMOVED]
16:37:29:343 GTK - hexdump(len=32): [REMOVED]
16:37:29:343 WPA: group state machine entering state SETKEYSDONE (VLAN-ID 0)
16:37:29:343 wpa_driver_nl80211_set_key: ifindex=7 (p2p0) alg=2 addr=0x4b760 key_idx=1 set_tx=1 seq_len=0 key_len=32
16:37:29:343 nl80211: KEY_DATA - hexdump(len=32): [REMOVED]
16:37:29:343     broadcast key
16:37:29:343 nl80211: set_key failed; err=-22 Invalid argument)
16:37:29:344 WPA: group state machine entering state FATAL_FAILURE
16:37:29:344 p2p0: Flushing old station entries
16:37:29:344 nl80211: flush -> DEL_STATION p2p0 (all)
16:37:29:344 p2p0: Deauthenticate all stations
16:37:29:344 nl80211: send_mlme - da= ff:ff:ff:ff:ff:ff noack=0 freq=0 no_cck=0 offchanok=0 wait_time=0 fc=0xc0 (WLAN_FC_
16:37:29:344 nl80211: send_mlme -> send_frame
16:37:29:346 nl80211: send_frame - Use bss->freq=2437
16:37:29:357 nl80211: send_frame(freq=2437 bss->freq=2437) -> send_monitor
16:37:29:365 wpa_driver_nl80211_set_key: ifindex=7 (p2p0) alg=0 addr=(nil) key_idx=0 set_tx=0 seq_len=0 key_len=0
16:37:29:380 wpa_driver_nl80211_set_key: ifindex=7 (p2p0) alg=0 addr=(nil) key_idx=1 set_tx=0 seq_len=0 key_len=0
16:37:29:381 wpa_driver_nl80211_set_key: ifindex=7 (p2p0) alg=0 addr=(nil) key_idx=2 set_tx=0 seq_len=0 key_len=0
16:37:29:395 wpa_driver_nl80211_set_key: ifindex=7 (p2p0) alg=0 addr=(nil) key_idx=3 set_tx=0 seq_len=0 key_len=0
16:37:29:396 hostapd_free_hapd_data(p2p0)
```

解决方法：

打开内核的.config 配置的宏：CONFIG_CRYPTO_ARC4=y

> *Are you sure your kernel has CONFIG_CRYPTO_ARC4=y?*

Indeed, the arc4 module is enabled as a module in the kernel but not deployed in my filesystem.

I added it and my error message is gone. I will test on several WiFi routers tonight.

Question:
How could I have know that this module is needed? I see not dependencies to the arc4 crypto. What did I miss?
I now needed to get deep into the source code to find the origin.
Are there any more crypto's I need for other routers?

--
Met vriendelijke groet / With kind regards,

Richard Knoop (richard.knoop at ibb.nl), Ingenieursburo Balvers BV
Tel +31 72 576 2552
Newtonstraat 27, 1704SB Heerhugowaard, Netherlands

### (6) hostapd.conf 设置隐藏 ssid 导致驱动挂掉

挂掉 log 如下：

```
random from internal pool - hexdump(len=16): [REMOVED]
mixed random - hexdump(len=32): [REMOVED]
GMK - hexdump(len=32): [REMOVED]
Get randomness[IF] add, start
atbm_upload_beacon: hidden ssid with ssid len 0 not supported
------------[ cut here ]------------
: len=32 entropyWARNING: CPU: 0 PID: 102 at /usr/lchome/yuzhihuang/Mstar/IPC_I
Modules linked in: atbm602x_wifi_usb(O) cfg80211 usb_storage ehci_hcd usbcore
CPU: 0 PID: 102 Comm: hostapd Tainted: G        O    3.18.30 #27
[<c000faad>] (unwind_backtrace) from [<c000e2e3>] (show_stack+0xb/0xc)
[<c000e2e3>] (show_stack) from [<c0015c71>] (warn_slowpath_common+0x45/0x60)
[<c0015c71>] (warn_slowpath_common) from [<c0015ce3>] (warn_slowpath_null+0xf/
=0
random from [<c0015ce3>] (warn_slowpath_null) from [<bf939f45>] (atbm_bss_info
os_get_random - [<bf939f45>] (atbm_bss_info_changed [atbm602x_wifi_usb]) from
hexdump(len=32):[<bf911e01>] (ieee80211_bss_info_change_notify [atbm602x_wifi_
 [REMOVED]
rand[<bf9206b9>] (ieee80211_config_beacon [atbm602x_wifi_usb]) from [<bf921711
om_mix_pool - he[<bf921711>] (ieee80211_start_ap [atbm602x_wifi_usb]) from [<b
[<bf8f88fb>] (nl80211_start_ap [cfg80211]) from [<c019a88f>] (genl_rcv_msg+0x1
[<c019a88f>] (genl_rcv_msg) from [<c019a179>] (netlink_rcv_skb+0x2f/0x64)
[<c019a179>] (netlink_rcv_skb) from [<c019a6ff>] (genl_rcv+0x15/0x26)
[<c019a6ff>] (genl_rcv) from [<c0199d53>] (netlink_unicast+0xcb/0x15e)
```

原因：

```
# Send empty SSID in beacons and ignore probe request frames that do not
# specify full SSID, i.e., require stations to know SSID.
# default: disabled (0)
# 1 = send empty (length=0) SSID in beacon and ignore probe request for
#     broadcast SSID
# 2 = clear SSID (ASCII 0), but keep the original length (this may be required
#     with some clients that do not support empty SSID) and ignore probe
#     requests for broadcast SSID
ignore_broadcast_ssid=0
```

设置隐藏 ssid 的参数是 ignore_broadcast_ssid。

上图是 hostapd.conf 对参数的说明，设置成 1，传给驱动的参数 ssid 长度直接为 0，而 ATBM 驱动如果遇到传入的 ssid_length=0,会导致驱动判断为出错。

所以需要将这个参数配置为 2，只是将 beacon 包中的 ssid 字段写 0，ssid_legnth 还是不变的，这样子驱动才能正常工作。

### (7) 执行了命令，hostapd 重载配置无效

Hostapd 重定义了信号的执行函数：

```
#ifndef CONFIG_NATIVE_WINDOWS

    eloop_register_signal(SIGHUP, handle_reload, interfaces);
    eloop_register_signal(SIGUSR1, handle_dump_state, interfaces);
#endif /* CONFIG_NATIVE_WINDOWS */
    eloop_register_signal_terminate(handle_term, interfaces);

#ifndef CONFIG_NATIVE_WINDOWS
    openlog("hostapd", 0, LOG_DAEMON);
#endif /* CONFIG_NATIVE_WINDOWS */
```

重载配置的命令为：

Kill(hostapd_pid,SIGHUP);

Hostapd_pid : hostapd 运行起来的进程号

修改 src/ap/hostapd.c 文件的 hostapd_reload_config 函数：

```
    if (newconf == NULL)
        return -1;

    hostapd_clear_old(iface);

    oldconf = hapd->iconf;
    iface->conf = newconf;

    for (j = 0; j < iface->num_bss; j++) {
        hapd = iface->bss[j];
        hapd->iconf = newconf;
    /*
        hapd->iconf->channel = oldconf->channel;
        hapd->iconf->acs = oldconf->acs;
        hapd->iconf->secondary_channel = oldconf->secondary_channel;
        hapd->iconf->ieee80211n = oldconf->ieee80211n;
        hapd->iconf->ieee80211ac = oldconf->ieee80211ac;
        hapd->iconf->ht_capab = oldconf->ht_capab;
        hapd->iconf->vht_capab = oldconf->vht_capab;
        hapd->iconf->vht_oper_chwidth = oldconf->vht_oper_chwidth;
        hapd->iconf->vht_oper_centr_freq_seg0_idx =
            oldconf->vht_oper_centr_freq_seg0_idx;
        hapd->iconf->vht_oper_centr_freq_seg1_idx =
            oldconf->vht_oper_centr_freq_seg1_idx;
    */
        hapd->iconf->channel = newconf->channel;
        hapd->iconf->acs = newconf->acs;
        hapd->iconf->secondary_channel = newconf->secondary_channel;
        hapd->iconf->ieee80211n = newconf->ieee80211n;
        hapd->iconf->ieee80211ac = newconf->ieee80211ac;
        hapd->iconf->ht_capab = newconf->ht_capab;
        hapd->iconf->vht_capab = newconf->vht_capab;
        hapd->iconf->vht_oper_chwidth = newconf->vht_oper_chwidth;
        hapd->iconf->vht_oper_centr_freq_seg0_idx =
            newconf->vht_oper_centr_freq_seg0_idx;
        hapd->iconf->vht_oper_centr_freq_seg1_idx =
            newconf->vht_oper_centr_freq_seg1_idx;

        hapd->conf = newconf->bss[j];
        hostapd_reload_bss(hapd);

    } « end for j=0;j<iface->num_bss;... »

    hostapd_config_free(oldconf);
```

## (8) hostapd 运行起来，手机连接上了但是没法建立 TCP 连接

并且出现如下错误：

```
  * aid=1
  * listen_interval=10
  * flags set=0x6 mask=0x6
p2p0: STA b0:e1:7e:13:af:a7 IEEE 802.11: binding station to interface 'p2p0'
nl80211: p2p0[4]: set_sta_vlan(b0:e1:7e:13:af:a7, ifname=p2p0[4], vlan_id=0)
nl80211: Set STA flags - ifname=p2p0 addr=b0:e1:7e:13:af:a7 total_flags=0x5 flags_or=0x5 flags_and=0xfffffff5 authorize
p2p0: STA b0:e1:7e:13:af:a7 RADIUS: starting accounting session 03708603-00000000
IEEE 802.1X: Ignore STA - 802.1X not enabled or forced for WPS
hostapd_new_assoc_sta: reschedule ap_handle_timer timeout for b0:e1:7e:13:af:a7 (1 seconds - ap_max_inactivity)
nl80211: Event message available
nl80211: Drx Event 19 (NL80211_CMD_NEW_STATION) received for p2p0
nl80211: New station b0:e1:7e:13:af:a7
ap_handle_timer: b0:e1:7e:13:af:a7 flags=0x80a3 timeout_next=0
p2p0: Station b0:e1:7e:13:af:a7 has been active 0s ago
ap_handle_timer: register ap_handle_timer timeout for b0:e1:7e:13:af:a7 (4 seconds)
ieee80211 phy0: Multicast delivery timeout.
dnsmasq-dhcp: DHCPDISCOVER(p2p0) b0:e1:7e:13:af:a7
dnsmasq-dhcp: DHCPOFFER(p2p0) 192.168.0.131 b0:e1:7e:13:af:a7
dnsmasq-dhcp: DHCPDISCOVER(p2p0) b0:e1:7e:13:af:a7
dnsmasq-dhcp: DHCPOFFER(p2p0) 192.168.0.131 b0:e1:7e:13:af:a7
dnsmasq-dhcp: DHCPDISCOVER(p2p0) b0:e1:7e:13:af:a7
dnsmasq-dhcp: DHCPOFFER(p2p0) 192.168.0.131 b0:e1:7e:13:af:a7
dnsmasq-dhcp: DHCPREQUEST(p2p0) 192.168.0.131 b0:e1:7e:13:af:a7
dnsmasq-dhcp: DHCPACK(p2p0) 192.168.0.131 b0:e1:7e:13:af:a7 HUAWEI_Mate_10_Pro-513400
ap_handle_timer: b0:e1:7e:13:af:a7 flags=0x80a3 timeout_next=0
p2p0: Station b0:e1:7e:13:af:a7 has been active 0s ago
ap_handle_timer: register ap_handle_timer timeout for b0:e1:7e:13:af:a7 (7 seconds)
ap_handle_timer: b0:e1:7e:13:af:a7 flags=0x80a3 timeout_next=0
p2p0: Station b0:e1:7e:13:af:a7 has been inactive too long: 1 sec, max allowed: 1
  Polling STA
nl80211: send_mlme - da= b0:e1:7e:13:af:a7 noack=0 freq=0 no_cck=0 offchanok=0 wait_time=0 fc=0x248 (WLAN_FC_STYPE_NULL
```

查看了 hostapd.conf 配置如下：

interface=p2p0

ctrl_interface=/var/run/hostapd

ctrl_interface_group=0

driver=nl80211

ssid=lyx

hw_mode=g

channel=11

macaddr_acl=0

auth_algs=1

ignore_broadcast_ssid=0

```
rsn_pairwise=CCMP
ap_max_inactivity=1
```

该配置缺少，加密类型以及密码就配置上加密算法。
正常配置如下：

```
interface=wlan0
#bridge=br0
ctrl_interface=/tmp/hostapd/
ctrl_interface_group=0
driver=nl80211
ieee80211n=1

#ht_capab=[HT40+]
#beacon_int=200
ssid=Sstar_ATBM_TEST
hw_mode=g
channel=1
macaddr_acl=0
auth_algs=1
wpa_group_rekey=0
ignore_broadcast_ssid=0
wpa=2
wpa_passphrase=12345678
wpa_key_mgmt=WPA-PSK
wpa_pairwise=CCMP
rsn_pairwise=CCMP
```

# 3 BRIDGE MDOE

## 3.1 内核需要打开如下宏用于使能 iwconfig，iwpriv，iwlist 命令

CONFIG_CFG80211_WEXT=y
CONFIG_WIRELESS=y
CONFIG_WIRELESS_EXT=y
CONFIG_WEXT_CORE=y
CONFIG_WEXT_PROC=y
CONFIG_WEXT_PRIV=y

## 3.2 功能说明

a) Altobeam6022 是单天线 WIFI，所以必须保证 wpa_supplicant 连接的路由器所在的信道和 hostapd 是在同一个信道。
b) 桥接启动成功，wpa_supplicant 连接上路由器，路由器分配的 IP 地址是作用在 br0 接口上的，此时启动 hostapd,手机连接上 hostapd,手机分配的 IP 地址也是路由器分配的。如果路由器可以上网，手机连接上 hostapd 后，正常也是可以上网的，如果不能上网就有问题。

## 3.3 启动流程

### (1) 加载 cfg80211.ko 见 STA MODE 部分

### (2) 加载 atbm_wifi.ko 见 STA MODE 部分

编译驱动的时候驱动要支持 bridge,需要把 hal_apollo 目录下的 Makefile 里面把 bridge 宏打开

```
ccflags-y += -DWIFI_FW_DOWNLOAD
# Extra IE for probe response from upper layer is needed in P2P GO
# For offloading probe response to FW, the extra IE must be included
# in the probe response template
ccflags-y += -DATBM_PROBE_RESP_EXTRA_IE
ccflags-y += -DCONFIG_ATBM_APOLLO_DEBUG
#ccflags-y += -DCONFIG_MAC80211_BRIDGE
#ccflags-y += -DIPV6_FILTERING
#ccflags-y += -DCONFIG_ATBM_APOLLO_BH_DEBUG
#ccflags-y += -DCONFIG_ATBM_APOLLO_WSM_DEBUG
#ccflags-y += -DCONFIG_ATBM_APOLLO_WSM_DUMPS
#ccflags-y += -DCONFIG_ATBM_APOLLO_WSM_DUMPS_SHORT
#ccflags-y += -DCONFIG_ATBM_APOLLO_TXRX_DEBUG
#ccflags-y += -DCONFIG_ATBM_APOLLO_TX_POLICY_DEBUG
#ccflags-y += -DCONFIG_ATBM_APOLLO_STA_DEBUG
#ccflags-y += -DCONFIG_ATBM_APOLLO_DUMP_ON_ERROR
#ccflags-y += -DCONFIG_ATBM_APOLLO_ITP
ccflags-y += -DCONFIG_ATBM_APOLLO_TESTMODE
# use the mac addr in file :"/data/.mac.info"
ccflags-y += -DCUSTOM_FEATURE_MAC
#ccflags-y += -DTEST_RF_POWER
```

如果这个宏没有打开，在设置的时候会出现如下问题：

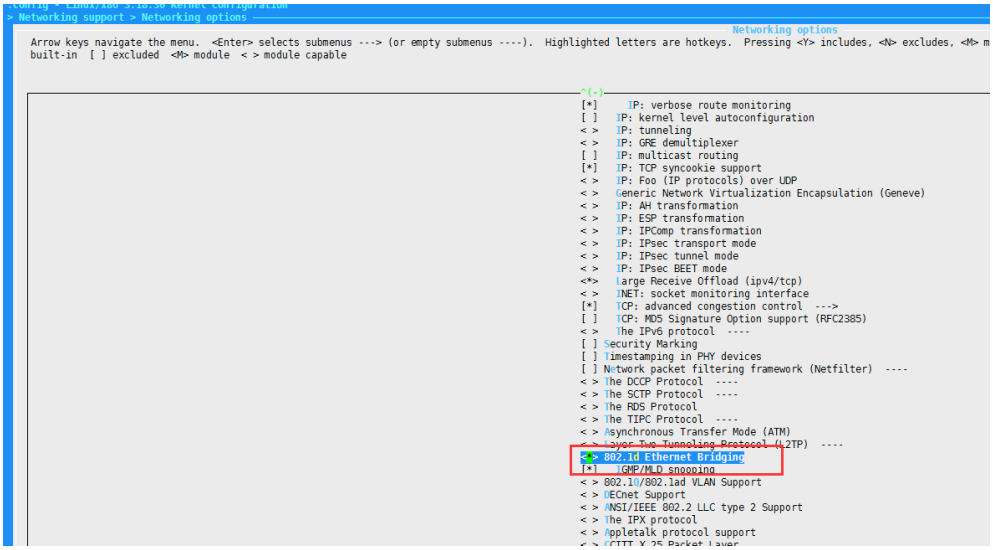
```
[/ext/demo]##
[/ext/demo]## ./brctl addif br0 wlan0
can't add wlan0 to bridge br0: Operation not supported
[/ext/demo]##
```

**(3) 加载 llc.ko**

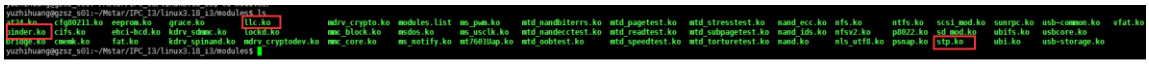
**(4) 加载 stp.ko**

**(5) 加载 bridge.ko**

a) 如果编译内核的时候已经将 802.1d 编译进内核则不需要这 3~5 步骤




b) 如果编译内核的时候将该选项选择为 M，编译为模块就需要 3~5 步骤
选择为“M”编译内核，最终生成的 llc.ko、stp.ko、bridge.ko 会在内核根目录的 modules 目录里面




**(6) 启动 wlan0 接口**

ifconfig wlan0 up

**(7) 启动 wlan1/p2p0 接口**

ifconfig [ wlan1 | p2p0 ] up

**(8) 设置网桥接口 br0**

ifconfig wlan0 0.0.0.0
ifconfig [ wlan1 | p2p0 ] 0.0.0.0
brctl addbr br0
brctl addif br0 wlan0
brctl addif br0 [ wlan1 | p2p0 ]

brctl setfd br0 0

正常 log 应如下图所示：

```
msc313:/opt/conf#brctl addbr br0
msc313:/opt/conf#brctl addif br0 wlan0
RTW: rtw_ndev_notifier_call(wlan0) state:20
RTW: rtw_ndev_notifier_call(wlan0) state:21
device wlan0 entered promiscuous mode
msc313:/opt/conf#brctl addif br0 wlan1
RTW: rtw_ndev_notifier_call(wlan1) state:20
RTW: rtw_ndev_notifier_call(wlan1) state:21
device wlan1 entered promiscuous mode
msc313:/opt/conf#brctl setfd br0 0
brctl: invalid argument 'setfd' to 'brctl'
msc313:/opt/conf#
msc313:/opt/conf#
msc313:/opt/conf#ifconfig br0 up
br0: port 2(wlan1) entered forwarding state
br0: port 2(wlan1) entered forwarding state
br0: port 1(wlan0) entered forwarding state
br0: port 1(wlan0) entered forwarding state
```

### (9) 启动网桥接口 br0

ifconfig br0 up

### (10) 启动 wpa_supplicant

wpa_supplicant.conf 的内容如下：

```
[/ext/demo]## cat run/wpa_supplicant.conf
ctrl_interface=/ext/demo/run/wpa_supplicant
update_config=1

#network={
#        ssid="ASUS"
#        psk="12345678"
#}

network={
        ssid="abc"
        psk="12345678"
}

#network={
#        ssid="tp123456"
#        psk="12345678"
#}

network={
        ssid="altobeamSZ_2.4g"
        psk="6270181130"
}
```

wpa_supplicant 执行命令为：

调试时候加上 -d 参数

wpa_supplicant -Dnl80211 -i wlan0 -c /ext/demo/run/wpa_supplicant.conf -bbr0 &

### (11) 启动 dhcpc

udhcpc –i br0

### (12) 启动 hostapd

hostapd.conf 内容如下：

```
[/ext/demo]## cat run/hostapd_brige.conf
interface=p2p0
bridge=br0
ctrl_interface_group=0

driver=nl80211

ssid=mt7601

hw_mode=g

channel=11

macaddr_acl=0

auth_algs=1

ignore_broadcast_ssid=0

wpa=3

wpa_passphrase=12345678

wpa_key_mgmt=WPA-PSK
wpa_pairwise=TKIP
rsn_pairwise=CCMP
[/ext/demo]## 
```

Hostapd 执行命令为：

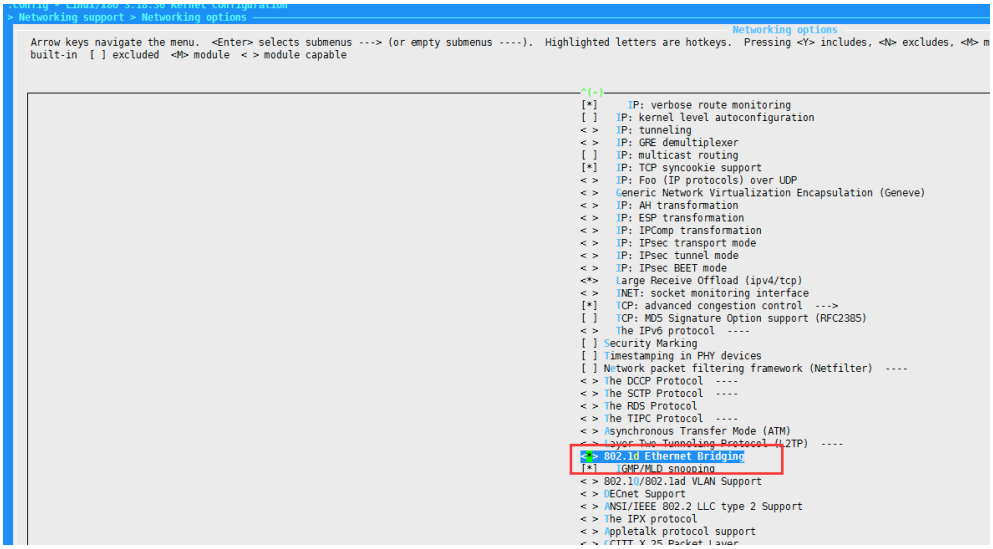
调试时候加上 –d 参数

hostapd -B /ext/demo/run/hostapd_brige.conf &

## 3.4 注意事项

### (1) 网桥接口 br0 起不来

内核不支持 802.1d




### (2) 使用 brctl 没法添加接口道 br0

错误如下：

```
[/ext/demo]##
[/ext/demo]## ./brctl addif br0 wlan0
can't add wlan0 to bridge br0: Operation not supported
[/ext/demo]##
```

altobeam 驱动中没有将 DCONFIG_MAC80211_BRIDGE 宏打开

```
76: ccflags-y += -DWIFI_FW_DOWNLOAD
77: # Extra IE for probe response from upper layer is needed in P2P GO
78: # For offloading probe response to FW, the extra IE must be included
79: # in the probe response template
80: ccflags-y += -DATBM_PROBE_RESP_EXTRA_IE
81: ccflags-y += -DCONFIG_ATBM_APOLLO_DEBUG
82: #ccflags-y += -DCONFIG_MAC80211_BRIDGE
83: #ccflags-y += -DIPV6_FILTERING
84: #ccflags-y += -DCONFIG_ATBM_APOLLO_BH_DEBUG
85: #ccflags-y += -DCONFIG_ATBM_APOLLO_WSM_DEBUG
86: #ccflags-y += -DCONFIG_ATBM_APOLLO_WSM_DUMPS
87: #ccflags-y += -DCONFIG_ATBM_APOLLO_WSM_DUMPS_SHORT
88: #ccflags-y += -DCONFIG_ATBM_APOLLO_TXRX_DEBUG
89: #ccflags-y += -DCONFIG_ATBM_APOLLO_TX_POLICY_DEBUG
90: #ccflags-y += -DCONFIG_ATBM_APOLLO_STA_DEBUG
91: #ccflags-y += -DCONFIG_ATBM_APOLLO_DUMP_ON_ERROR
92: #ccflags-y += -DCONFIG_ATBM_APOLLO_ITP
93: ccflags-y += -DCONFIG_ATBM_APOLLO_TESTMODE
94: # use the mac addr in file :"/data/.mac.info"
95: ccflags-y += -DCUSTOM_FEATURE_MAC
96: #ccflags-y += -DTEST_RF_POWER
```

### (3) 使用桥接以后发现获取 IP 地址很慢

通过执行 brctl show 查看正常和非正常获取 IP 地址的区别：

正常获取 IP 地址：

```
bridge name     bridge id               STP enabled     interfaces
br0             8000.6095ce60f39b       no              p2p0
                                                        wlan0
msc313:/tmp#
```

获取 IP 地址时间很长：

```
root@jabsco:/tmp# brctl show
bridge name     bridge id               STP enabled     interfaces
br0             8000.dc2919007645       yes             p2p0
                                                        wlan0
root@jabsco:/tmp#
```

上网查了资料：

**如何处理STP开启时终端Ping网关不通或获取IP地址慢的问题？**

终端设备（如服务器、网管等）不支持运行STP协议，但是由于接入交换机的STP功能是开启状态，那么接口状态在Up/Down上不停变换时，接口在30秒时间内才能进入转发状态，导致终端Ping网关不通或者获取IP地址慢。

为了解决上述问题，需要在连接终端的接口上配置边缘端口或关闭STP功能。

从可用性和安全性考虑，建议把端口指定为边缘端口，因为在连接的终端设备出现环路时，边缘端口可以自动切换为非边缘端口，自动启动该接口STP破环功能。

解决方法：

brctl stp br0 off

### (4) 将 p2p0 加入桥接以后想修改网口模式，但是修改失败

```
~ #
~ # /home/iwconfig p2p0 mode manager
[atbm_log]:ieee80211_netdev_ioctrl:cmd err
Error for wireless request "Set Mode" (8B06) :
    SET failed on device p2p0 ; Invalid argument.
~ #
```

P2p0 启动了 ap 模式，将 hostapd kill 掉以后查询 p2p0 也是处于 ap 模式必须要将 p2p0 移出桥接就能够修改模式成功。

```
~ #
~ # iw dev
phy#0
        Interface p2p0
                ifindex 4
                type AP
        Interface eth3
                ifindex 3
                type managed
~ # brctl show
bridge name     bridge id               STP enabled     interfaces
eth2            8000.dc2919c69af3       no              eth3
                                                        p2p0
~ #
~ # iw p2p0 set type managed
command failed: Device or resource busy (-16)
~ #
~ # ifconfig eth2 down
eth2: port 2(p2p0) entered disabled state
~ # brctl delbr eth2
device p2p0 left promiscuous mode
eth2: port 2(p2p0) entered disabled state
device eth3 left promiscuous mode
eth2: port 1(eth3) entered disabled state
~ #
~ # iw p2p0 set type managed
[atbm_log]:p2p0: is not sta mode
[atbm_log]:br0_netdev_open()-1116: dev_get_by_name(br0) failed2!~ #
~ #
~ # iw dev
phy#0
        Interface p2p0
                ifindex 4
                type managed
        Interface eth3
                ifindex 3
                type managed
~ #
```

原因是桥接模式下内核限制了不让网口修改模式：

```
    return -EOPNOTSUPP;

/* if it's part of a bridge, reject changing type to station/ibss */
if ((dev->priv_flags & IFF_BRIDGE_PORT) &&
    (ntype == NL80211_IFTYPE_ADHOC ||
     ntype == NL80211_IFTYPE_STATION ||
     ntype == NL80211_IFTYPE_P2P_CLIENT))
    return -EBUSY;
```

# 4 Monitor mode

## 4.1 内核需要打开如下宏用于使能 iwconfig，iwpriv，iwlist 命令

CONFIG_CFG80211_WEXT=y
CONFIG_WIRELESS=y
CONFIG_WIRELESS_EXT=y
CONFIG_WEXT_CORE=y
CONFIG_WEXT_PROC=y
CONFIG_WEXT_PRIV=y

## 4.2 启动 monitor 接口：

**1、ifconfig wlan0 down**
**2、iwconfig wlan0 mode monitor**
**3、ifconfig wlan0 up**
**4、ifconfig wlan0 0.0.0.0**
**5、iwconfig wlan0 channel [1,13] //** 选择一个信道监听

## 4.3 切换监听信道

**iwconfig wlan0 channel [1,13] //** 选择一个信道

## 4.4 关闭 monitor 接口：

**1、ifconfig wlan0 down**
**2、iwconfig wlan0 mode manager**

## ***CONTACT INFORMATION***

AltoBeam (China) Inc.

Address: B808, Tsinghua Tongfang Hi-Tech Plaza, Haidian, Beijing, China 100083

Tel: (8610) 6270 1811

Fax: (8610) 6270 1830

Website: www.altobeam.com

Email: support@altobeam.com

## ***DISCLAIMER***

Information in this document is provided in connection with AltoBeam products. No license, express or implied, by estoppels or otherwise, to any intellectual property rights is granted by this document. Except as provided in AltoBeam's terms and conditions of sale for such products, AltoBeam assumes no liability whatsoever, and AltoBeam disclaims any express or implied warranty, relating to sale and/or use of AltoBeam products including liability or warranties relating to fitness for a particular purpose, merchantability, or infringement of any patent, copyright or other intellectual property right.

AltoBeam may make changes to specifications and product descriptions at any time, without notice.

Designers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." AltoBeam reserves these for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them.

Unauthorized use of information contained herein, disclosure or distribution to any third party without written permission of AltoBeam is prohibited.

AltoBeam™ is the trademark of AltoBeam. All other trademarks and product names are properties of their respective owners.

Copyright © 2007~2020 AltoBeam, all rights reserved